SpoofWatch

Teaching computers to spot market cheating

A complete guide in plain English: what problem this solves, how it is built, what goes in, what came out, the code that matters, the screens, and how to run and explain it.

Built on real NASDAQ order book data from 21 June 2012

214 automated tests - every number traced back to a saved file

SIMULATED DATA - this does not claim to catch real-world cheating

What is in this guide

#	Section	The question it answers
1	The problem	What is the cheating, and why is it hard to catch?
2	How it is built	What are the parts and how do they fit together?
3	What goes in	What data is used, and what does every number mean?
4	How it was done	What were the steps, and what went wrong on the way?
5	What came out	What did it prove, and what did it not?
6	The important code	Which lines matter, and what does each one do?
7	The screens	What am I looking at, and how do I drive it?
8	Running it	Which commands, in what order?
9	Questions	What will I be asked, and what do I say?

Words you will need

Only nine. Everything else in this guide is plain English.

Word	What it means, simply
Order book	The live list of every buy and sell order waiting at each price. Like a queue board.
Spoofing	Placing a big order you never intend to complete, to trick others about demand.
Agent	A computer program that takes actions and learns from the results.
Reinforcement learning (RL)	Training by reward instead of by examples. You never show it the right answer - you only say 'that went well' or 'that went badly', and it works the rest out.
Episode	One run of the simulation. Here: 200 decisions covering 2,000 market events.
Training run (or 'seed')	Training the same program again from a different random starting point. We do it three times and report the spread, because a result that only happens once is luck, not a finding.
Recall	Of all the real cheating, what share did we catch? 1.0 means we caught everything.
Precision	Of everything we accused, what share was actually cheating? 1.0 means no false alarms.
F1 score	Precision and recall combined into one number between 0 and 1. Higher is better.
False positive rate	How often we accuse someone innocent. Lower is better.

> **The one thing to understand before anything else**
> This project has two halves, and they ended very differently.
>
> The **attacker half worked**: computer programs taught themselves to cheat, purely by chasing profit. Nobody told them how.
>
> The **detector half did not**, in an interesting way. It got very good scores - but we then discovered the test was far too easy, because the answer was accidentally hidden in the data we were feeding it. We found that ourselves and published it. That honesty is the point of the project, not a weakness in it.

1. The problem

What is spoofing?

Imagine you want to sell a share. Nobody is paying much for it right now. So you do this:

- You place a **huge buy order** at a slightly lower price. You do not want it to go through.
- Everyone else sees a big buyer on the board and thinks demand is rising. The price drifts up.
- You **sell** into that higher price.
- You **cancel** your huge buy order before anyone can take you up on it.

You never intended to buy anything. The big order was bait. That is spoofing, and it is illegal - it is banned by name in US law (Dodd-Frank Act, section 747).

So why not just catch it?

Because the action on its own is completely normal. Placing a big order and cancelling it is something honest traders do all day long - they change their mind, the price moves, they manage their risk. What makes it illegal is the **intention**, and no computer file records what someone intended.

Exchanges today mostly use a simple rule: "big order, cancelled quickly, flag it". That does catch cheats. It also flags a huge number of innocent people - in our tests, **82 out of every 100** orders it flagged were legitimate.

What this project asked

> **The question**
> Can a computer **learn** to spot market cheating it has never seen before, without accusing honest traders - and without anyone telling it what cheating looks like?

There were three things to prove. Here is how each one ended:

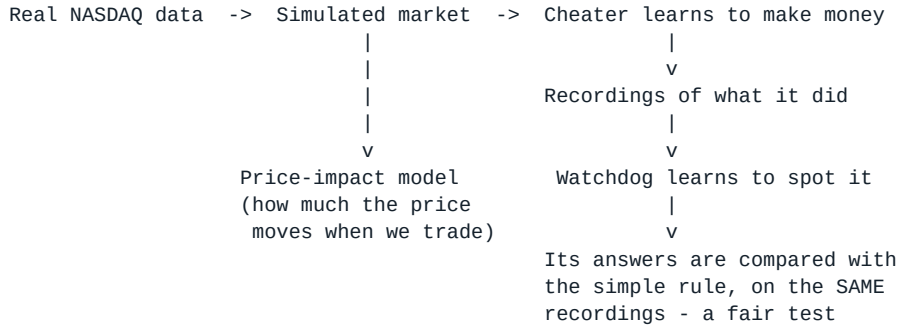
What we wanted to prove	How it ended
1. The programs really learn to cheat (they are not just exploiting a bug in our simulation)	**Mostly yes.** They make real money in our main simulation. In a second, different simulation they lose money - so the result depends on our assumptions.
2. The detector can catch cheating styles it has never seen	**No at first, then yes** - but we later found the test was too easy, so the 'yes' means less than it looks.
3. The detector does not accuse innocent traders	**Yes.** It wrongly flags 0.2 out of 100, where the simple rule flags 82 out of 100.

2. How it is built

Four parts. Think of it as a stage with actors on it.

Part	Think of it as	What it actually is
1. The market	The stage	A real day of NASDAQ trading, replayed order by order, with a model that makes the price move when our agent trades.
2. The cheaters	The villains	Programs told only one thing: make money. Never told what spoofing is. If they start spoofing, it is because they worked out that it pays.
3. The watchdog	The detective	A program that watches one trader and says 'suspicious' or 'fine' at every moment. Never given the rule for what spoofing looks like.
4. The simple rule	The old guard	"Big order, cancelled fast." This is roughly what real exchanges use today. It is the thing we have to beat.

How information flows



The cheaters are frozen before the watchdog starts learning. That way the cheaters cannot adapt to the detective, and the comparison stays fair.

Why there are two different markets

Any result that only appears in one simulation is not really a result about cheating - it is a result about that simulation. So everything is checked twice.

Market	How the price moves	What it is for
Replay market	Real historical prices, plus a model of how our own orders push the price	The main results.
Synthetic market	Prices appear naturally from simple simulated traders buying and selling from each other	A second opinion. Built to match real statistics, never tuned to make spoofing pay.

> **An important honest result**
> Spoofing **makes money** in the first market and **loses money** in the second one, where a fake order moves the price 4 to 9 times less. We do not know which simulation is closer to reality. So the correct statement is: spoofing pays *under our price model* - not 'spoofing pays'.

3. What goes in

The raw data

Thing	Value
Where it comes from	LOBSTER, a public dataset of real order book activity
Which market and day	NASDAQ, 21 June 2012 - **one single day** (this limits everything)
Which companies	Apple, Microsoft, Google, Intel, Amazon
How much	About 2.1 million individual order messages
What we throw away	The first 30,000 messages. The market is strange right after it opens - Intel's order book was 12 times thinner than usual.
One 'episode'	200 decisions by the agent, covering 2,000 market events

How much of it was used

Where	How many episodes	What it is for
Training	150 per source, about 16 sources	Teaching the programs
Testing	50 per source	Choosing settings and checking progress
Held out	50 per source	Never used for any decision - the real exam
Each new attack test	50 per company, all five companies	Run once, result committed in advance
Profit measurements	50 episodes, 3 training runs	Checking the money is real

What the detective sees: 46 numbers, every moment

This is the most important part of the whole project, so it is worth going slowly.

At every step, the watchdog is handed 46 numbers. **Forty** of them describe the public order book - what anyone watching the market could see. **Six** describe the specific trader being watched. Two of those six turned out to be a problem.

Numbers 0 to 39: the public order book

Ten price levels on each side. For each level we record the price and the size of orders waiting there.

Numbers	What it is	Why it is written this way
prices	How far this price sits from the middle of the market	Measured in 'spreads' rather than dollars, so a $585 share and a $27 share can be compared fairly.
sizes	How many shares are waiting at this price	Compared against how deep the book normally is, so 'a big order' means the same thing on a busy stock and a quiet one.

The watched trader's own orders are mixed into these sizes, exactly as a real market feed would show them. This block does not say *who* placed what.

Numbers 40 to 45: the trader being watched

These only exist because our simulation knows which orders belong to whom. A real anonymous market feed could not give you these.

Number	What it tells you	Does it give away the answer?
40	How big their resting order is	A little - it says a big order exists
41	How long their oldest order has been sitting	No
42	How much they just placed	No
43	How much they just cancelled	No
44	**Did they just buy or sell?** (+1 bought, -1 sold, 0 neither)	**Yes - together with 45**
45	**Which side is their big order on?** (buy side or sell side)	**Yes - together with 44**

What we are trying to predict

We mark an order as cheating if, **while that order was sitting there, its owner traded the other way**. That is our definition of spoofing for this project.

Now look again at numbers 44 and 45

Our definition is: *"traded the other way while their order was sitting there"*.

Number 44 says **which way they traded**. Number 45 says **which side their order is on**.

Put those two together and you have simply restated the definition. So we tried it: just compare those two numbers, with **no learning at all**. It scores **0.997 out of 1.0**.

In other words, we had accidentally been handing the detective the answer sheet.

4. How it was done, step by step

Step 1 - make the market push back

Historical prices do not react to someone who was not there. If our fake order cannot move the price, spoofing cannot work and there is nothing to study.

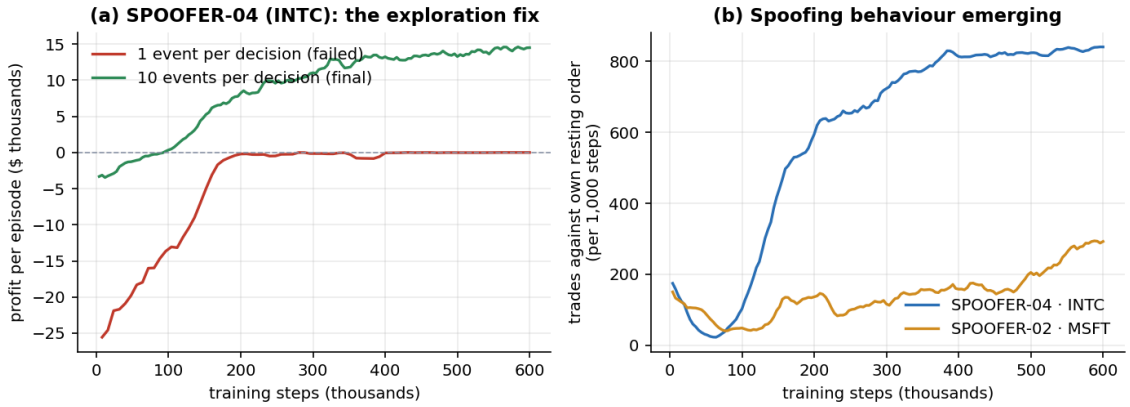
So we measured, from the real data, **how much the price actually moves** when buying and selling gets lopsided. That measurement is then used to move the price when our agent trades. It is measured per company, not guessed.

We tried a second way of measuring this and **threw it away**: it implied the price would jump 44 to 58 spreads, which is not believable. We kept the rejected method in the project as a record.

Step 2 - let the cheating appear by itself

The cheater programs get one instruction: make money. They have six possible actions - buy, sell, place a big buy order, place a big sell order, cancel, do nothing. The word 'spoofing' appears nowhere in their training.

Cheating appeared on Microsoft and Intel only. On Apple, Google and Amazon the gap between buy and sell prices is too wide for it to pay, so those programs never bothered. That is a finding in itself.



A cheating program learning, over 600,000 practice steps. Profit (left) climbs as it discovers the trick; the right-hand chart shows it placing more and more large fake orders as it learns they pay.

Four times the simulation was cheated instead

Four times we saw big profits and got excited, and four times it turned out the program had found a hole in our simulation rather than a real strategy. Each was only caught by asking *where exactly* the money came from.

What it found	What we saw	How we closed it
Free shares	Buying far below the real market price; $253,000 a go	Made the agent's own orders move the price against it too
Swinging back and forth	Tiny cost, unlimited benefit; $240,000 a go	Price impact now builds up over time, and the benefit is capped at the order's size
Quitting while behind	All 31 early exits happened right before a loss	No early exits allowed
Trading at the open	The order book is unusually thin just after opening	Skip the first 30,000 messages

Step 3 - train the detective, and watch it fail

The watchdog gets points for being right: **+1** for catching cheating, **-1** for missing it, **-2** for a false accusation, **0** for correctly staying quiet. The -2 is deliberate: wrongly accusing someone is worse than missing someone.

On cheating styles it had seen before, it did well. On **new** styles it failed badly - it caught 22 out of 100 where the old simple rule caught 89.

Step 4 - work out why

> **The moment the project turned around**
> The detective was not broken. **Our labelling was.**
>
> We had been marking the *entire time* a cheating order sat there as 'cheating happening now' - including the period **before** the person had actually done anything wrong.
>
> So the detective learned the wrong job. Instead of learning *'this is cheating happening'*, it learned *'this person looks like a cheater'*. The proof: it flagged **99%** of one cheater's orders *before* that cheater had done anything, and caught only **14%** of another cheater's actual cheating moments.

Three changes fixed it:

- **Mark the act, not the person.** Only start calling it cheating from the moment they actually trade the other way.
- **Judge the moment in front of you** rather than trying to predict the future.
- **Show it cheaters with different timing** during training, so it cannot rely on one rhythm.

Catching new cheating styles went from **22 out of 100** to **98 out of 100**.

Step 5 - check whether we had actually achieved anything

We had not, as much as it looked. Two checks showed why:

- Just comparing numbers 44 and 45 - **no learning whatsoever** - scores 0.997. Our clever detective reached a bar that needs no cleverness at all.
- A much simpler standard learning method matches it, and trains in **minutes** instead of hours.

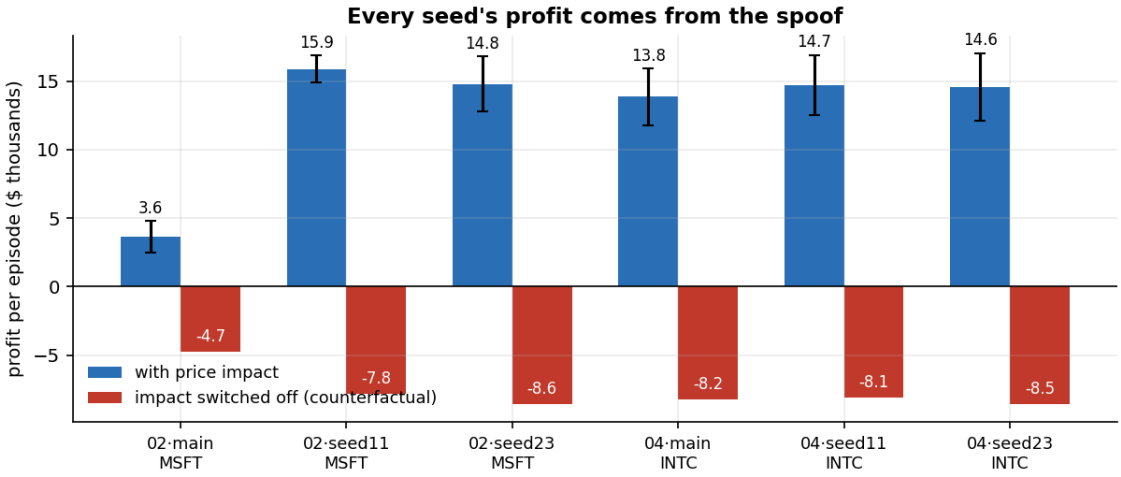
5. What came out

The cheaters - this part genuinely worked

Program	Company	Profit per episode	With price-impact turned off	In the other market
SPOOFER-04	Intel	**+$14,368**	-$8,279	-$3,270
SPOOFER-02	Microsoft	**+$11,431**	-$7,037	-$2,930
The other three	Apple, Google, Amazon	about zero - never learned to cheat	-	-

Read the fourth column. It is the proof. If we switch off the part of the simulation that lets fake orders move the price, the exact same trading **loses money**. So the profit comes from the trickery, not from being good at trading.

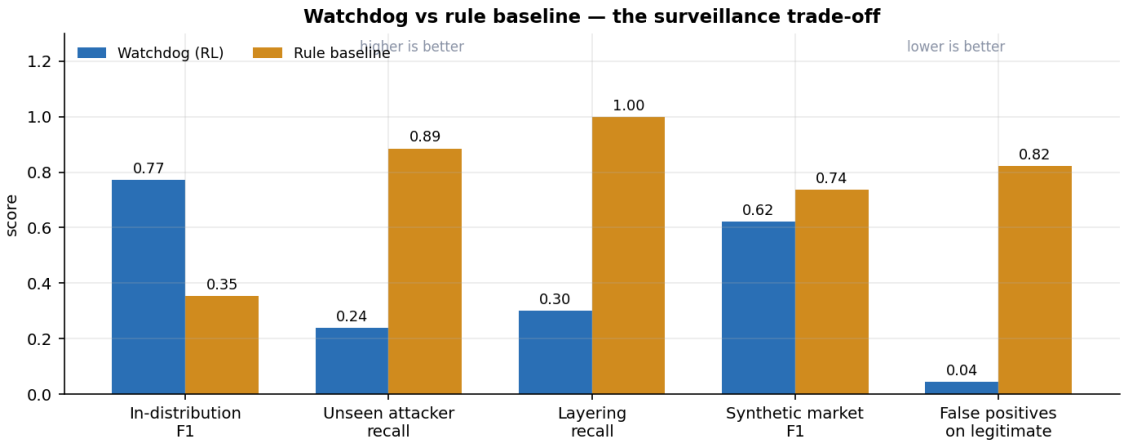
Each result is from three separate training runs. Microsoft's result swings a lot between runs (+$3,628 to +$15,886). We report that rather than hiding it behind an average.



Each cheating program trained three separate times. Intel (left) lands in the same place every time. Microsoft (right) swings a lot between runs - we show that rather than hiding it behind an average.

The detective - three versions

What we measured	First try	After the fix	Final version	Old simple rule
Catching cheating it had seen before	0.75	0.98	**0.98**	0.35
Wrongly accusing innocent traders	3 in 100	0.3 in 100	**0.2 in 100**	82 in 100
Catching new style 1 (late burst)	22 in 100	48 in 100	**98 in 100**	89 in 100
Catching new style 2 (slow drip)	7 in 100	39 in 100	**99 in 100**	74 in 100
Catching new style 3 (reversal)	14 in 100	45 in 100	**98 in 100**	96 in 100
Catching new style 4 (layering)	30 in 100	-	**99 in 100**	100 in 100



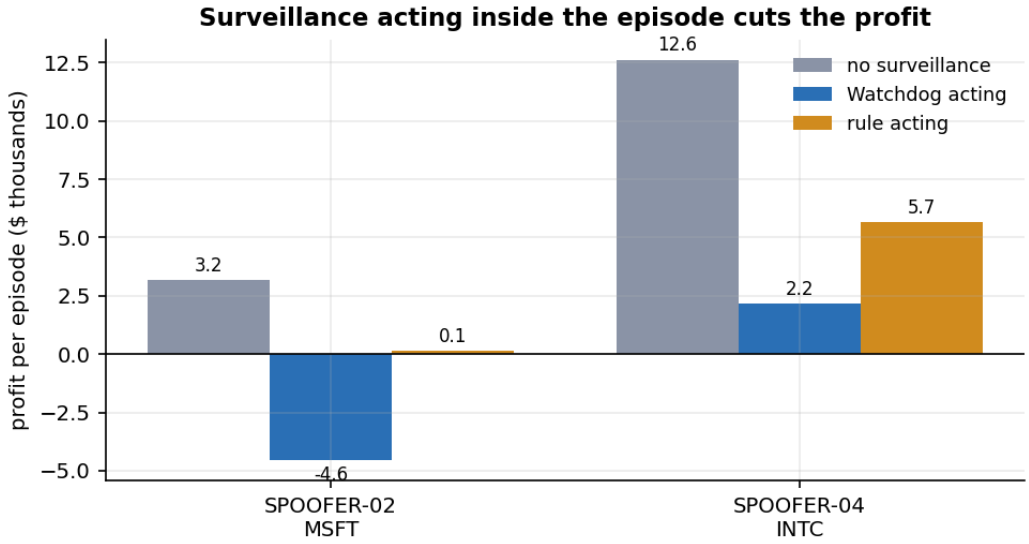
The detective against the old rule across every test. Tall green is good on the left-hand charts, short is good on the right-hand one (false accusations).

But here is the ceiling

Everything above has to be read next to this table. These are all scored on the same data.

Method	Score (out of 1.0)	How long it takes to build
Just compare numbers 44 and 45	**0.997**	No training at all
Write the definition out as a rule	0.986	No training at all
A standard learning method	0.986	A few minutes
Our reinforcement learning detective	0.984	A million training steps
A simple statistical model	0.528	Seconds
The old 'big and cancelled' rule	0.352	Two settings

Our clever method is *fourth*, behind something that requires no learning whatsoever. That is the honest headline.



What happens to the cheater's profit when surveillance is switched on inside the simulation. The detective removes more of it than the old rule does - and bothers innocent traders far less often.

The fair test: take away the answer sheet

So we removed numbers 40 to 45 entirely and tried to detect cheating using **only the public order book** - what a real outside observer would actually have.

Method (public data only)	Caught	False alarms	Score
Flag absolutely everything	100 in 100	100 in 100	0.218
Simple book-imbalance rule	9 in 100	9 in 100	0.099
Simple statistical model	38 in 100	8 in 100	**0.392**
Pattern-finding model	13 in 100	2 in 100	0.203
Sequence-learning model	3 in 100	1 in 100	0.052

0.392, down from 0.997. That is the honest size of the problem once you stop handing over the answer. It still beats flagging everything (0.218), so there is a real signal in the public data - just a modest one.

Three things this table hides unless you look carefully

1. The most sophisticated model is the *best* on familiar data and the *worst* on new data - it memorised instead of learning. If we had only reported familiar-data scores, it would have looked like the winner.

2. It works on Intel and Microsoft and is *worse than flagging everything* on the other three companies. The single average hides two opposite results.

3. None of the models' confidence levels are trustworthy on new data - they are worse than simply guessing the average rate.

6. The important code

Six pieces. Each is short, and each one does something the whole project depends on.

6.1 The definition of cheating - two lines

`evaluation/baseline_detector.py, line 51`

```
    @property
    def manipulative(self) -> bool:
        return self.opposite_trades > 0
```

In words: an order counts as cheating if, while it was sitting there, its owner traded the other way at least once. `opposite_trades` simply counts those trades. That is the whole definition - everything else in the project is measured against it.

6.2 The six numbers about the watched trader - where the problem lives

`evaluation/rollout.py, line 37`

```
    return np.array([
        np.log1p((bid + ask) / depth),                              # 40 how big their order is
        min(age / 200.0, 5.0),                                      # 41 how long it has been sitting
        np.log1p(placed / depth),                                   # 42 what they just placed
        np.log1p(cancelled / depth),                                # 43 what they just cancelled
        float(trade_dir),                                           # 44 DID THEY BUY OR SELL?
        np.sign(imbalance) * np.log1p(abs(imbalance) / depth),      # 45 WHICH SIDE IS THEIR ORDER?
    ], dtype=np.float32)
```

In words: line 44 records which way they traded. Line 45 records which side their big order is on. Our definition of cheating is 'traded the other way while their order was sitting there'. These two lines are that sentence, written as numbers.

6.3 The two-line detector that exposed the problem

`scripts/label_observability.py, line 40`

```
    "opposite-side signature":
        lambda X: (np.sign(X[:, RESTING_IMBALANCE]) * np.sign(X[:, TRADE_DIR])) < 0,
```

In words: `np.sign` turns a number into +1, -1 or 0. If the order side and the trade direction point *opposite* ways, multiplying them gives a negative number. So: *if negative, flag it*.

There is nothing to train and no settings to choose. It scores **0.997**. This single line is why the project's detector results have to be described as a ceiling rather than a success.

6.4 The fix that changed everything

`watchdog/dataset.py, line 156`

```
    y = np.zeros(len(src["y"]), dtype=bool)          # start with 'nothing is cheating'
    off = src["ep_offsets"]
    for ep, o in orders_from_array(src["orders"]):   # look at every big order
        if not o.manipulative:
            continue                                 # skip the honest ones
        a = off[ep]
        trades = src["X"][a + o.placed_step:a + o.removed_step, TRADE_COL]
        hit = np.flatnonzero(trades == -o.side)      # first trade the OTHER way
        if hit.size:
```

```
            y[a + o.placed_step + hit[0]:a + o.removed_step] = True
    return y
```

In words, line by line. Start by assuming nothing is cheating. Go through every big order and ignore the honest ones. For a cheating order, look at what its owner did while it was sitting there. Find the **first** moment they traded the other way - that is `trades == -o.side`, where the minus sign means 'the opposite side'. Mark cheating from **that** moment, not from when the order was placed.

Why it mattered. The old version marked the whole period, so the detective could score well just by recognising who the cheater was. The new version only gives credit for reacting once something has actually happened. It keeps 87% of the marks - the change is not *how many*, but *which ones*.

6.5 The old rule we had to beat

`evaluation/baseline_detector.py, line 60`

```
    def flags(self, order: OrderRecord) -> bool:
        return (order.depth_mult >= self.n_mult              # at least N times normal size
                and order.removed_by == "cancel"             # cancelled, not completed
                and order.lifetime_events <= self.m_events)  # within M market events
```

In words: big, cancelled, and quick. Two settings, chosen using training data only (at least 2x normal size, cancelled within 1,000 market events). It catches almost every cheat - and also flags 82 out of every 100 honest large orders, which is why no one could actually use it.

6.6 How the detective is scored

`watchdog/watchdog_env.py, line 61`

```
    if manipulative_order_resting:
        reward = self.tp_reward if flag else -self.fn_cost    # +1 caught it, -1 missed it
    else:
        reward = -self.fp_cost if flag else 0.0               # -2 false alarm, 0 stayed quiet
```

In words: a false accusation costs twice as much as a miss. That is on purpose - investigating an innocent trader is expensive and unfair, so we want the detective to keep quiet when it is unsure.

A failure worth knowing about. At first the detective learned to *never* flag anything, because staying silent scored better than guessing. The cause was not the scoring - it was that the 46 numbers were on wildly different scales. Putting them on a common scale fixed it.

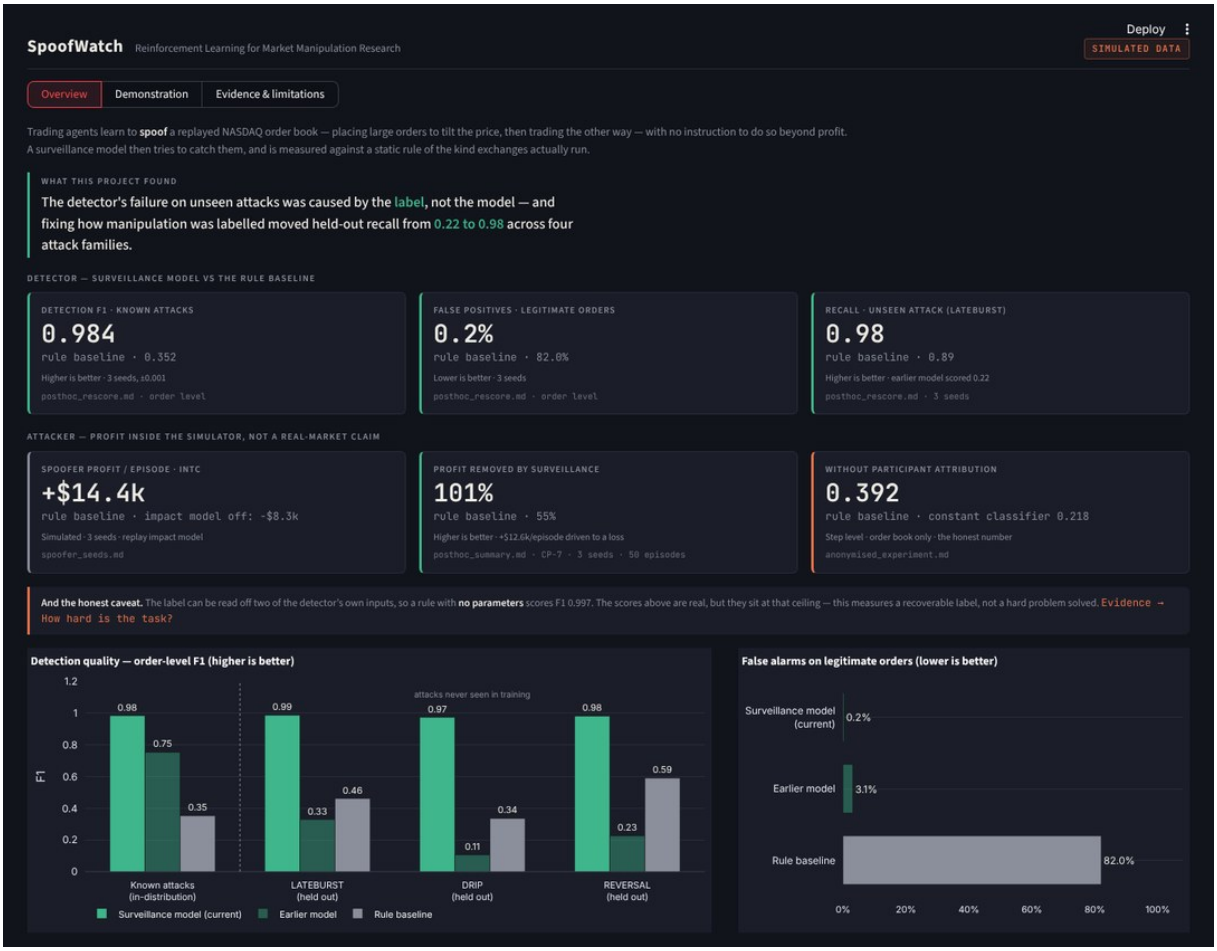
7. The screens

One app, three screens. Every number on screen is read from a saved results file. Nothing is calculated, trained or downloaded when you open it.

```
streamlit run app.py --server.fileWatcherType none
```

That last part is not optional. This project contains a folder called `watchdog`, and Streamlit happens to use a tool with the same name. Without the flag they clash and the app will not start. If anything goes wrong on the day, open the file `dashboard.html` instead - same numbers, no app needed.

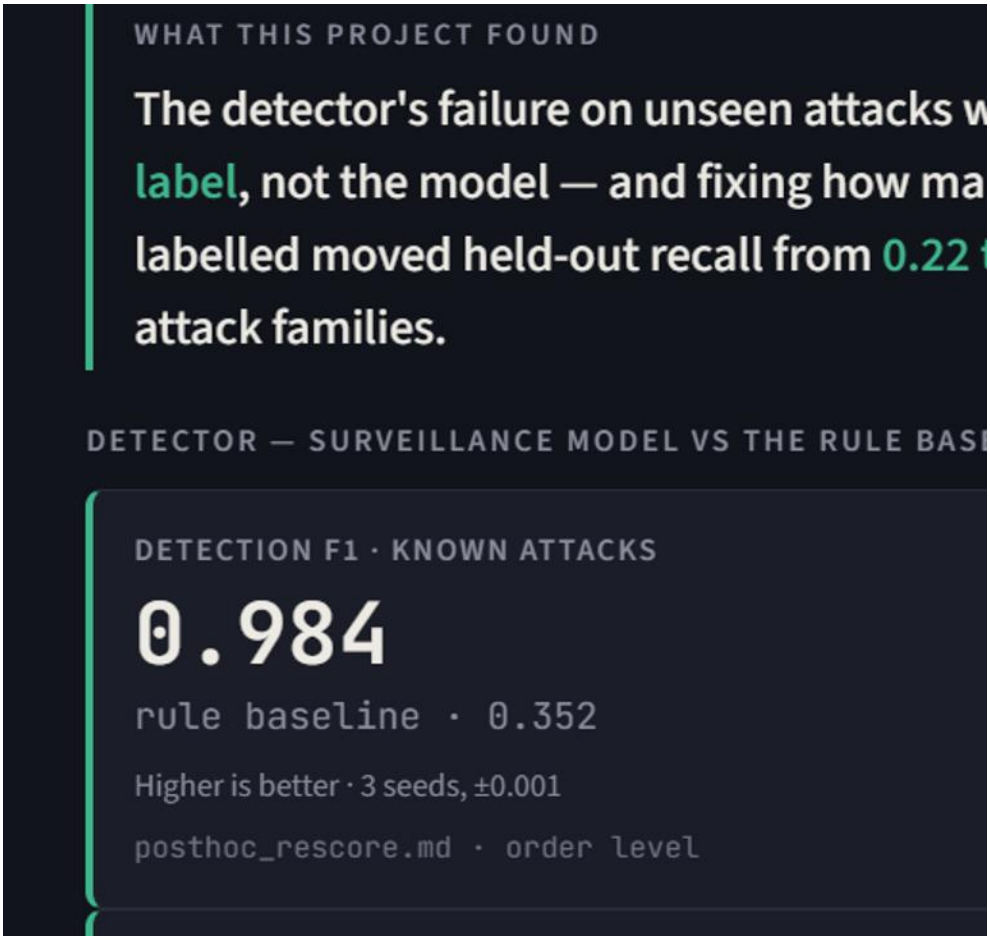
Screen 1 - Overview: the whole page



The entire first screen. Read it top to bottom: what the project is, what it found (green bar), three cards about the detective, three about the cheaters, the honest warning in orange, then two charts and the main finding.

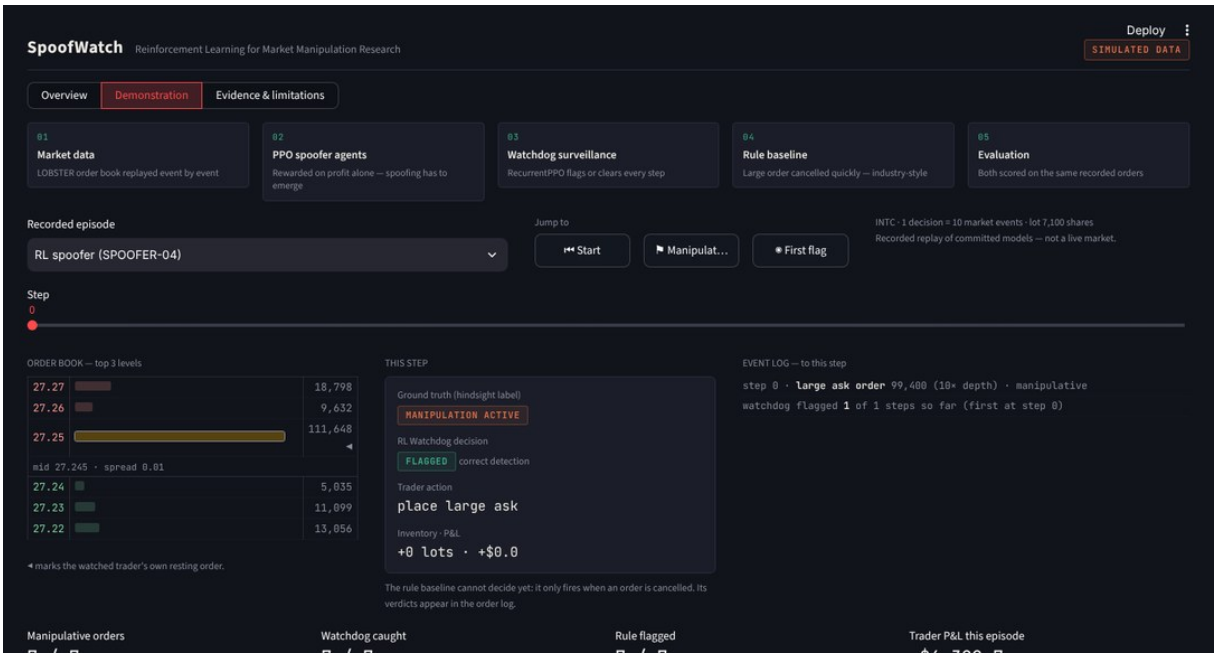
Why the cards are in two labelled groups. The cheaters' profit must never be mistaken for the detective's performance. They are different claims, so they sit in separate rows with their own headings.

Every card has four parts: the big number, the old rule's number underneath for comparison, a line telling you which direction is good, and the name of the file the number came from. That last part means anything on screen can be checked in one step.



A single card close up. 0.984 is ours, 0.352 is the old rule, 'higher is better' tells you how to read it, and posthoc_rescore.md is where it came from.

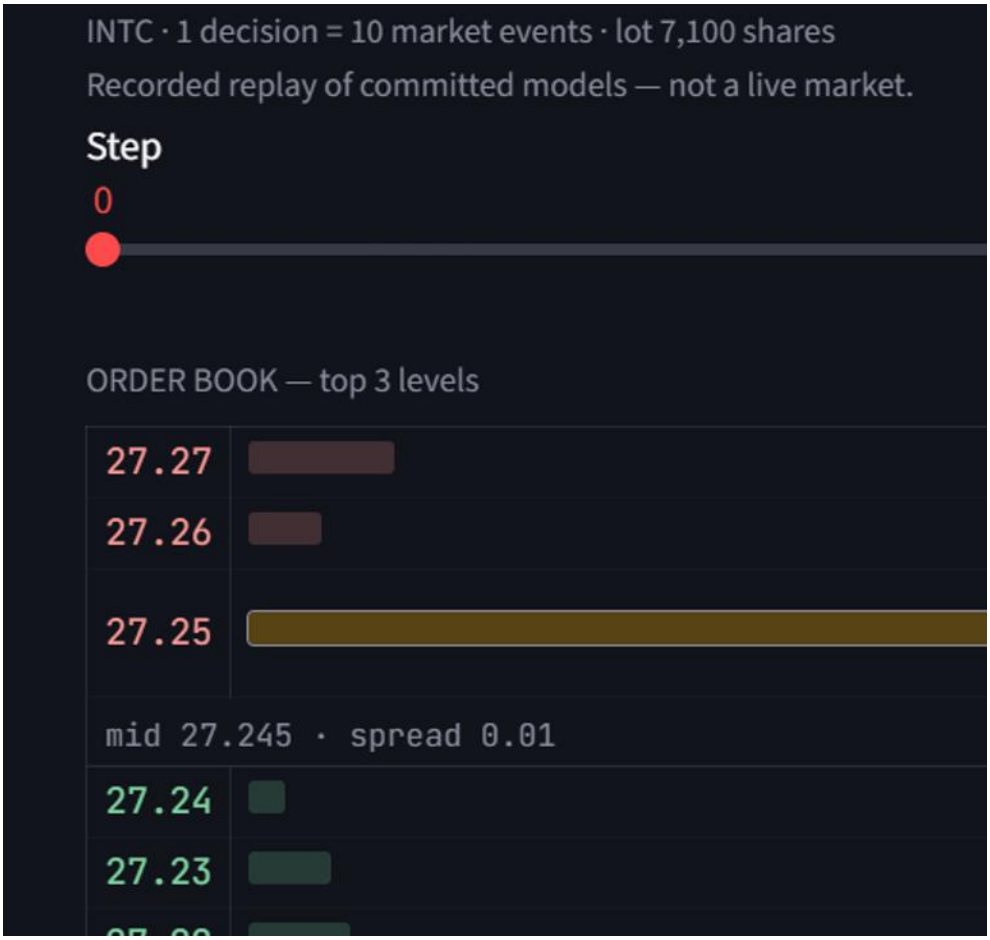
Screen 2 - Demonstration: watching one cheat happen



The whole demonstration screen. Along the top are the five stages of the project. Below that: pick a recording, jump to a moment, and step through it. The three panels show the order book, what is happening right now, and a running log.

How to drive it. Choose a recording from the dropdown, then use the three buttons - **Start**, **Manipulation** (jumps to the first cheating moment) and **First flag** (jumps to when the detective reacts). The slider moves one step at a time.

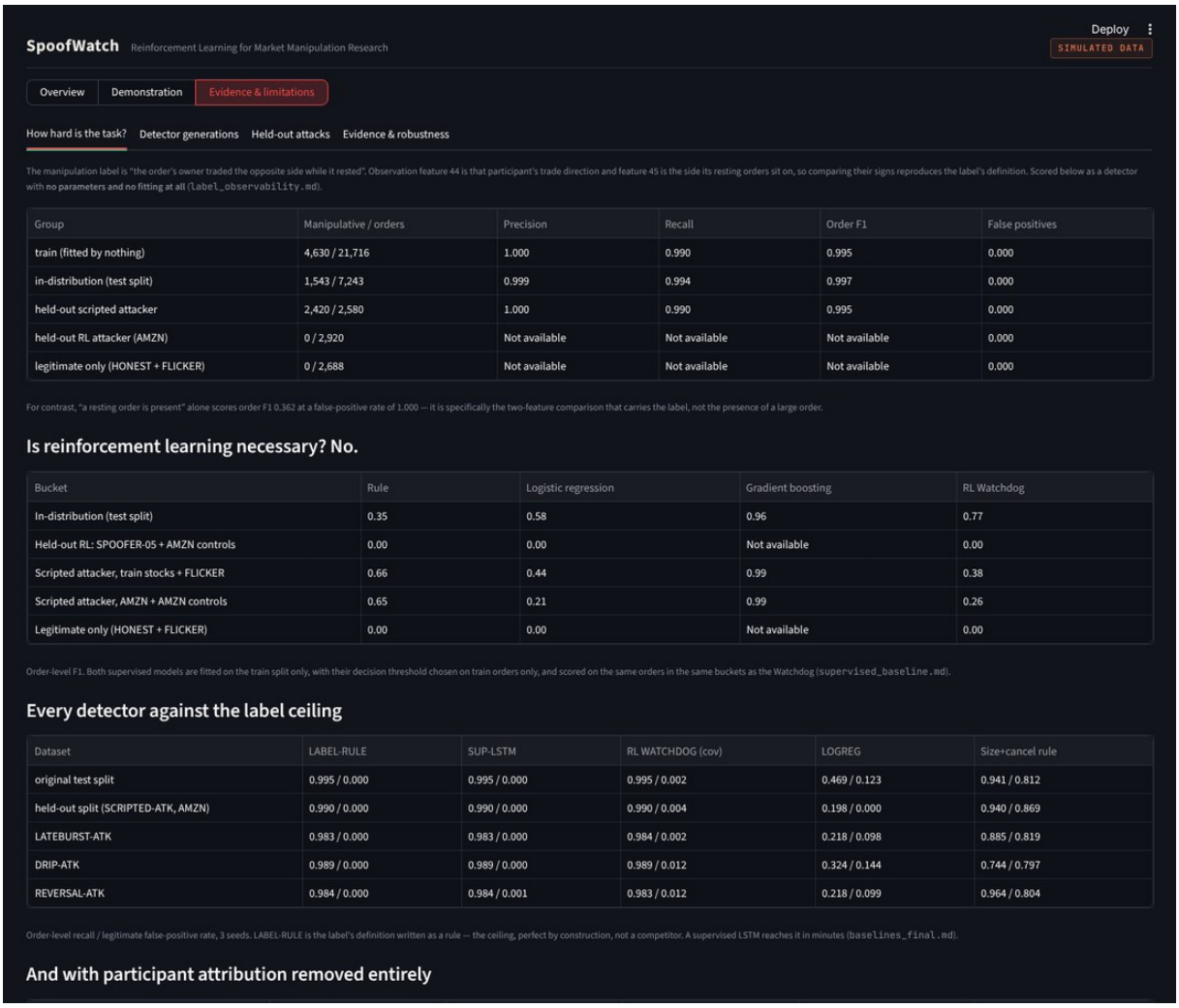
There are three recordings, and you should show at least two: a program that taught itself to cheat, a cheating style the detective had *never seen before*, and an **honest trader** who places big orders and cancels them without cheating. That last one is how you prove it does not accuse innocent people.



The order book close up. Red rows are people selling, green are people buying, and the number is the price. The wide orange bar is the watched trader's own fake order - many times bigger than anything around it. That is what spoofing looks like.

> **Say this sentence out loud when you demo**
> "This is a **recording** being replayed, not a live market." The screen says so too. Claiming it is live would be the quickest way to lose marks.

Screen 3 - Evidence: how hard is the task?



SpoofWatch Reinforcement Learning for Market Manipulation Research

Deploy

SIMULATED DATA

Overview | Demonstration | Evidence & limitations

How hard is the task? | Detector generations | Held-out attacks | Evidence & robustness

The manipulation label is "the order's owner traded the opposite side while it rested". Observation feature 44 is that participant's trade direction and feature 45 is the side its resting orders sit on, so comparing their signs reproduces the label's definition. Scored below as a detector with no parameters and no fitting at all (`label_observability.md`).

Group	Manipulative / orders	Precision	Recall	Order F1	False positives
train (fitted by nothing)	4,630 / 21,716	1.000	0.990	0.995	0.000
in-distribution (test split)	1,543 / 7,243	0.999	0.994	0.997	0.000
held-out scripted attacker	2,420 / 2,580	1.000	0.990	0.995	0.000
held-out RL attacker (AMZN)	0 / 2,920	Not available	Not available	Not available	0.000
legitimate only (HONEST + FLICKER)	0 / 2,688	Not available	Not available	Not available	0.000

For contrast, "a resting order is present" alone scores order F1 0.362 at a false-positive rate of 1.000 — it is specifically the two-feature comparison that carries the label, not the presence of a large order.

Is reinforcement learning necessary? No.

Bucket	Rule	Logistic regression	Gradient boosting	RL Watchdog
In-distribution (test split)	0.35	0.58	0.96	0.77
Held-out RL: SPOOFER-05 + AMZN controls	0.00	0.00	Not available	0.00
Scripted attacker, train stocks + FLICKER	0.66	0.44	0.99	0.38
Scripted attacker, AMZN + AMZN controls	0.65	0.21	0.99	0.26
Legitimate only (HONEST + FLICKER)	0.00	0.00	Not available	0.00

Order-level F1. Both supervised models are fitted on the train split only, with their decision threshold chosen on train orders only, and scored on the same orders in the same buckets as the Watchdog (`supervised_baseline.md`).

Every detector against the label ceiling

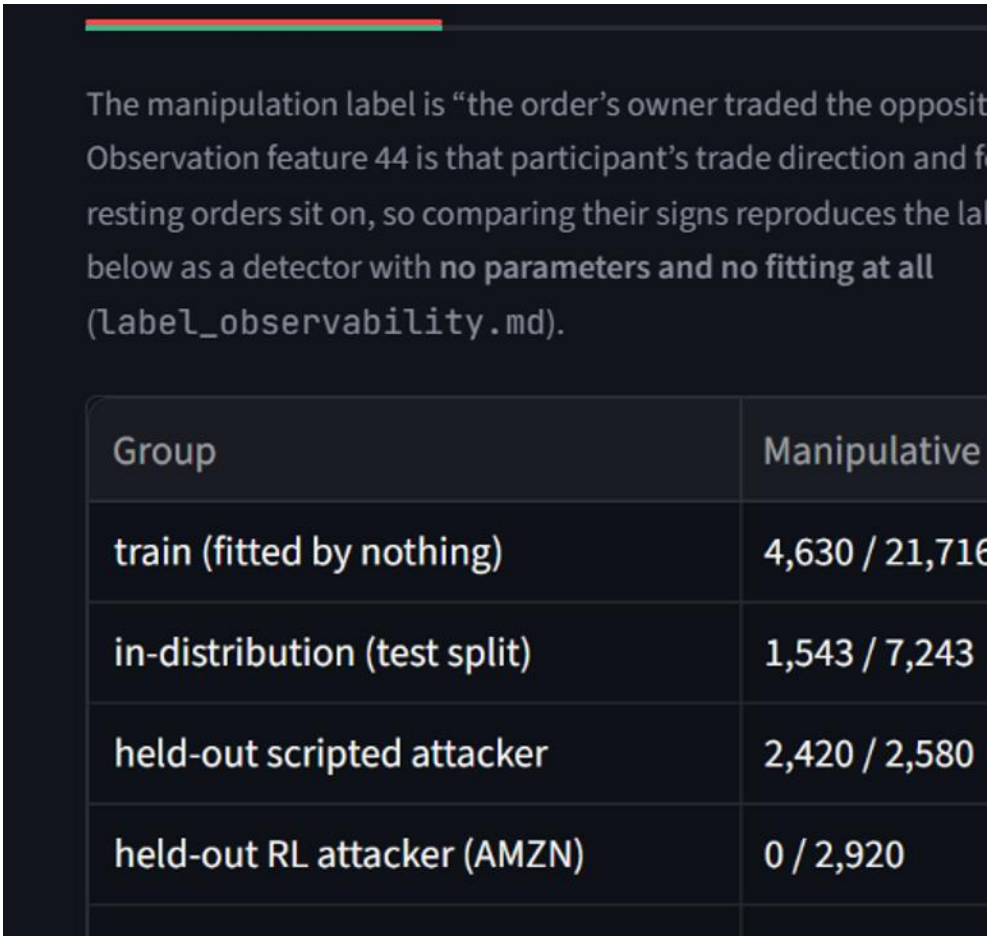
Dataset	LABEL-RULE	SUP-LSTM	RL WATCHDOG (cov)	LOGREG	Size+cancel rule
original test split	0.995 / 0.000	0.995 / 0.000	0.995 / 0.002	0.469 / 0.123	0.941 / 0.812
held-out split (SCRIPTED-ATK, AMZN)	0.990 / 0.000	0.990 / 0.000	0.990 / 0.004	0.198 / 0.000	0.940 / 0.869
LATEBURST-ATK	0.983 / 0.000	0.983 / 0.000	0.984 / 0.002	0.218 / 0.098	0.885 / 0.819
DRIP-ATK	0.989 / 0.000	0.989 / 0.000	0.989 / 0.012	0.324 / 0.144	0.744 / 0.797
REVERSAL-ATK	0.984 / 0.000	0.984 / 0.001	0.983 / 0.012	0.218 / 0.099	0.964 / 0.804

Order-level recall / legitimate false-positive rate, 3 seeds. LABEL-RULE is the label's definition written as a rule — the ceiling, perfect by construction, not a competitor. A supervised LSTM reaches it in minutes (`baselines_final.md`).

And with participant attribution removed entirely

The most important screen in the project, and all four of its tables. Top: the no-learning-required method scored as if it were a detector. Then: is the fancy method needed (no). Then: every method against the ceiling. Bottom: what happens when the giveaway numbers are taken away.

Why this tab comes first. You cannot judge our scores until you know what the bar is. Putting our own results first and this afterwards would be misleading, so we do it the other way round.

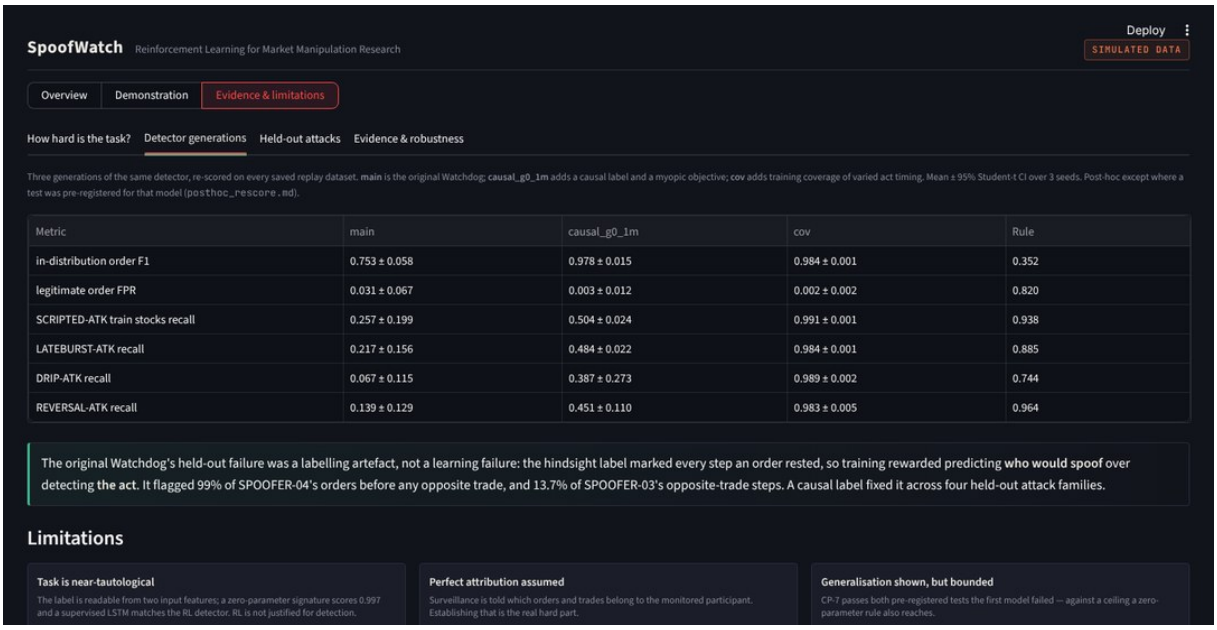


The manipulation label is “the order’s owner traded the opposit
Observation feature 44 is that participant’s trade direction and f
resting orders sit on, so comparing their signs reproduces the la
below as a detector with **no parameters and no fitting at all**
(`label_observability.md`).

Group	Manipulative
train (fitted by nothing)	4,630 / 21,716
in-distribution (test split)	1,543 / 7,243
held-out scripted attacker	2,420 / 2,580
held-out RL attacker (AMZN)	0 / 2,920

The first table close up. 'train (fitted by nothing)' is not a typo - that row really is a method with nothing to train.

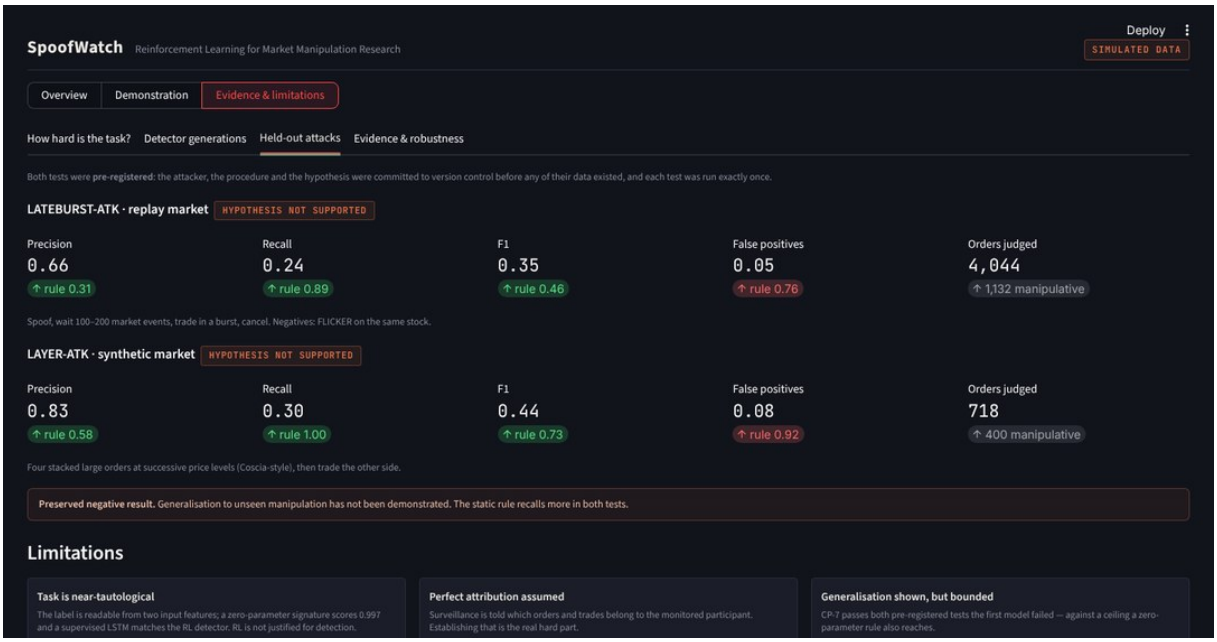
Screen 3, second tab - the three versions of the detective



The three detectives side by side, so you can see exactly what the labelling fix changed. Below the table is the explanation, and at the bottom are the six honest limitations.

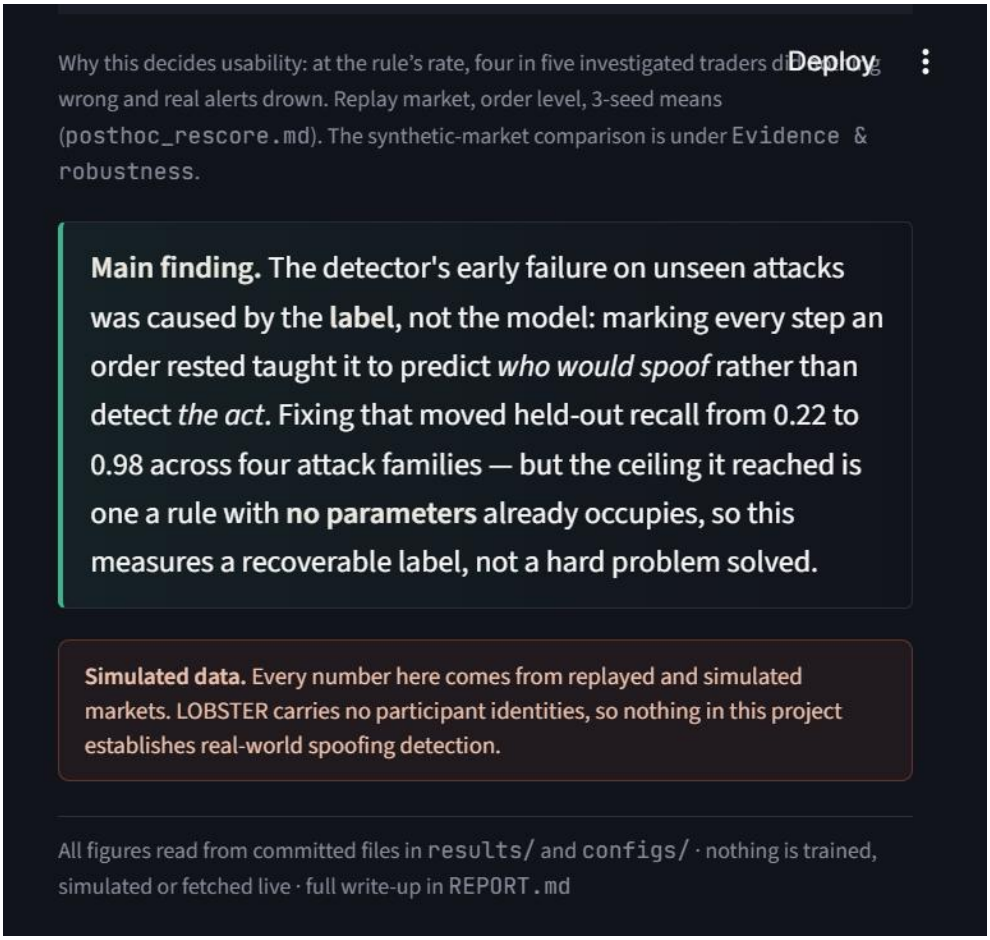
Read the LATEBURST row. 0.217, then 0.484, then 0.984. That single row is the story of the project: a failure, a diagnosis, and a fix.

Screen 3, third tab - the tests we committed to in advance



The two tests we wrote down before creating their data. Note the orange badges: HYPOTHESIS NOT SUPPORTED. These are the results that went against us, and they are shown exactly as they came out.

This is the screen to be proud of. We predicted the detective would beat the old rule on cheating it had never seen. It did not - it got 0.24 against the rule's 0.89. We published that unchanged. Later versions of the detective do much better, but those numbers appear on the previous tab, clearly labelled as coming after the fact.



The page ends with the main finding written out in full, and a red warning. Red is used for exactly one thing in this whole app - the reminder that all of it is simulated.

If a file is missing

The app does not crash. The affected card simply reads **"Not available"** and everything else still works - we tested this on purpose. One thing to remember: the app loads the files once when it starts, so if you change a file you must **restart** it. Refreshing the page is not enough.

8. Running it

First-time setup

```
pip install -r requirements.txt
pip install -r requirements-dashboard.txt     # needed for the app and charts
python data/download.py                       # about 600 MB of market data
```

> **You almost certainly do not need to train anything**
> Every trained program is already saved in the project - 79 files, about 15 MB. Someone downloading this project fresh can reproduce every result in this guide **without training for a single second**.

Check everything works

```
python -m pytest tests -q                     # should say 214 passed, about 2 minutes
streamlit run app.py --server.fileWatcherType none
```

Reproduce the results

```
python scripts/calibrate.py                   # measure how much the price moves
python watchdog/dataset.py --episodes 150     # record what the detective would see
python watchdog/evaluate_watchdog.py --tag cov    # the detective vs the old rule
python scripts/label_observability.py         # the ceiling - run this one
python watchdog/supervised_baselines.py train # is the fancy method needed? (no)
python watchdog/anonymised_experiment.py      # the fair test, answer sheet removed
python watchdog/posthoc_rescore.py            # every version on every dataset
```

Train from scratch, if you really want to

```
python training/train_spoofer.py SPOOFER-04 --timesteps 600000
python watchdog/train_watchdog.py --timesteps 400000 --norm-obs --ent-coef 0.01
```

Both of those flags on the second line are required. Without `--norm-obs` the detective learns to never flag anything at all. Without the other one it stops exploring too early and gets stuck.

A safety message you may see

```
$ python watchdog/drip_test.py
drip_test.py: refusing to overwrite results that already exist:
  results/drip_test.json
  results/drip_test.md
These record a run that was meant to happen once. Pass --rerun if you really
intend to replace them, or move them aside first.
```

Some tests are meant to be run **once only**, because we committed to the result in advance. The project now physically stops you from overwriting them by accident.

What you need to run it

Thing	What is needed
Machine	An ordinary laptop. No graphics card is required - everything here runs on the processor.
To just look at the results	Seconds. Open the app or the HTML file; nothing is computed.

Thing	What is needed
To re-check all the results	A few minutes per script. Nothing needs training.
To run the test suite	About 2 to 3 minutes for all 214 checks.
To train a cheater from scratch	Roughly 20 to 40 minutes for 600,000 practice steps.
To train a detector from scratch	Roughly 1 to 2 hours. You should not need to - they are saved.
Disk space	About 600 MB for the market data, plus 15 MB of saved programs.

Where to find things

File	What it is
README.md	The short overview - what was built and what it found
REPORT.md	The formal write-up: method, results, discussion, limitations
docs/TECHNICAL_MANUAL.md	The full manual - every command and every result
docs/FINAL_AUDIT.md	The audit: what was checked, what was broken, the verdict
docs/FEATURE_DICTIONARY.md	Every input number explained, and whether it leaks the answer
docs/DEMO_GUIDE.md	The minute-by-minute plan for a 10-minute presentation
docs/MODEL_COMPARISON.md	Every method compared fairly on the same data
docs/EXPERIMENT_PROTOCOL.md	The rules we set ourselves before running the tests

9. Questions you will be asked

Q1. Can this catch real market cheating?

No, and we never claim it can. The real data does not say *who* placed each order, and our detective needs that. So it cannot even be switched on against real market data - only the old simple rule can. There is also no public dataset anywhere with real cheating marked in it, so nobody could check the answer even if we ran it.

Q2. Your detector scores 0.98. Isn't that too good to be true?

Yes - which is exactly why we checked. Simply comparing two of the numbers we were feeding it, with **no learning at all**, scores 0.997. We had reached a bar that needs no cleverness. We found that ourselves and put it on the front page rather than hiding it.

Q3. Then why is 'reinforcement learning' in the project title?

For the detective, honestly, it should not be - a much simpler method matches it in minutes. Reinforcement learning earns its place on the **cheater** side, where there is no right answer to copy from and the program genuinely has to work the strategy out for itself. Our report says this plainly.

Q4. So what did the project actually achieve?

Two things. First, programs taught themselves to cheat profitably, and we proved the money comes from the trickery and not from lucky trading. Second, and more useful to anyone else: we showed that a **labelling mistake** had quietly turned 'spot the cheating' into 'spot the cheater', and that fixing it took the score from 22 out of 100 to 98 out of 100.

Q5. How do I know you did not just tune the tests until they passed?

Because we wrote down the test, the attacker and the expected result **before** creating any of that test's data, and saved that to version control with a timestamp. Anyone can check the order of events. Each test was run once. The first two failed, and we published that.

Q6. Was there a mistake in how you split training and testing data?

Yes, and we found and fixed it. Our test data overlapped our training data - 33 of 50 test runs shared market data with training. We fixed it by giving each split a separate stretch of the day, and then **measured** how much the mistake had flattered us: 0.031, which is smaller than the normal variation between training runs.

Q7. Is your simulation realistic?

Partly, and we list exactly where it is not. The price does react to our orders, using a model measured from real data. But historical traders cannot react to us, there is no queuing for position, no trading fees and no delays. Only one big order per side is allowed, which is why one cheating style had to be scripted by hand rather than learned.

Q8. How much do the results change between training runs?

A lot in one place, and we show it rather than hiding it. The Microsoft cheater's profit ranges from +$3,628 to +$15,886 across three runs. Intel is steady. Where a method has no randomness, the variation is exactly zero - that is correct, not a bug.

Q9. What happens if you take away the numbers that give the answer?

The score drops from 0.997 to **0.392**. That is the honest difficulty of the real problem. It still beats flagging everything, so there is genuine signal in public data - but it is modest, and we say so.

Q10. Why does that work for some companies and not others?

We do not know, and we say so. It works on Intel and Microsoft and fails on Apple, Google and Amazon. We tested two explanations and **both were wrong**: the cheater behaves identically on all five, and the disturbance it causes is actually *smaller* on the two where we succeed. With only five companies we cannot prove a cause.

Q11. What is the biggest weakness?

One day of data, five companies, and only two where cheating pays at all. That limits almost every conclusion here, and no amount of extra programming fixes it - it needs more data.

Q12. What would you do next?

Stop telling the detective which orders belong to whom, and make it work it out from public market activity alone. That is the change that turns this into a genuinely hard and realistic problem. Everything else is secondary until that is done.

Q13. How do I know the numbers in your report match your code?

The project checks itself. Fourteen automatic tests compare each headline number in the documents against the file that produced it - including the number of tests we claim to have. If a document goes out of date, the tests fail.

Q14. What do you need to run it?

An ordinary laptop. No graphics card. Looking at the results takes seconds because every trained program is already saved in the project. Only if you insisted on training from scratch would you need an hour or two.

Q15. Has anyone else reproduced this?

Yes. The results were produced on a Mac and re-run on a Windows machine. The detective's scores matched to three decimal places, and two complete test datasets were rebuilt from scratch and gave identical answers across 1,164 and 1,615 separate numbers.

The whole project in one paragraph

We built a simulated stock market out of one real day of NASDAQ trading. Into it we put computer programs whose only instruction was to make money - and they taught themselves to cheat, by placing big fake orders to push the price around. We proved the money came from the trickery by switching that effect off and watching the same trading lose money. We then built a second program to catch them. At first it failed on cheating styles it had not seen. The cause turned out to be our own **labelling**, not the program: we had been marking the whole time a fake order sat there, which taught it to recognise *people* rather than *actions*. Fixing that took it from catching 22 in 100 to 98 in 100. But we then discovered that simply comparing two of the numbers we were feeding it - with no learning at all - scores even higher. So the detective half is an honest negative result, and we published it that way.

> **The fair summary**
> This project is correct, repeatable and honest. Its weakness is not a mistake - it is a limit: one day of data. Its strength is that it found, measured and published the flaw in its own setup before anybody else could.